

YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

Project – Low Level Design (LLD)

on

**End-to-End Encryption for Manufacturing
Cloud Apps**

Student Name: Pranith Shetty B

Register Number: 23BCCE128

**Course: BCA in Cloud Computing, Cyber Security &
Ethical Hacking with IBM**

Industry Mentor: Mr. Shashank Guided by: Ms. Vijayalaxmi

S.No.	Section Title	Page No.
1	Introduction	1
1.1	Purpose of the Document	1
1.2	Scope of the Document	1
1.3	System Overview	1
2	System Architecture	2
2.1	Detailed Application Architecture	2
2.2	Module Interaction Flow	3
3	Module Design	4
3.1	Module Overview	4
3.2	Authentication Module (Login/Register)	4
3.3	Encryption Module	5
3.4	Decryption Module	6
3.5	Messaging Module	6
4	Database Design	7
4.1	Database Schema	7
4.2	Table Structures	7-9
4.3	Relationships	10
5	API Design	11
5.1	API Endpoints	11
5.2	Request and Response Format	12
6	Process Flow	14
6.1	System Workflow	14

S.No.	Section Title	Page No.
7	Session and Access Control	7
8	Error Handling and Validation	8
9	References	19

1. Introduction

1.1 Purpose of the Document

This Low-Level Design (LLD) document provides a detailed technical blueprint for building the End-to-End Encryption for Manufacturing Cloud Applications project.

The purpose of this document is to:

- Explain the internal working of each module
- Define how data flows between components
- Specify the database structure and API endpoints
- Guide developers to implement the system correctly

1.2 Scope of the Document

This document covers the complete technical design of the secure messaging system including system architecture, module design for login and encryption, database tables and relationships, API endpoints with request and response formats, process flows for key operations, session management, and error handling. The scope is limited to the encryption-based messaging system for manufacturing environments and does not cover hardware or network infrastructure.

1.3 System Overview

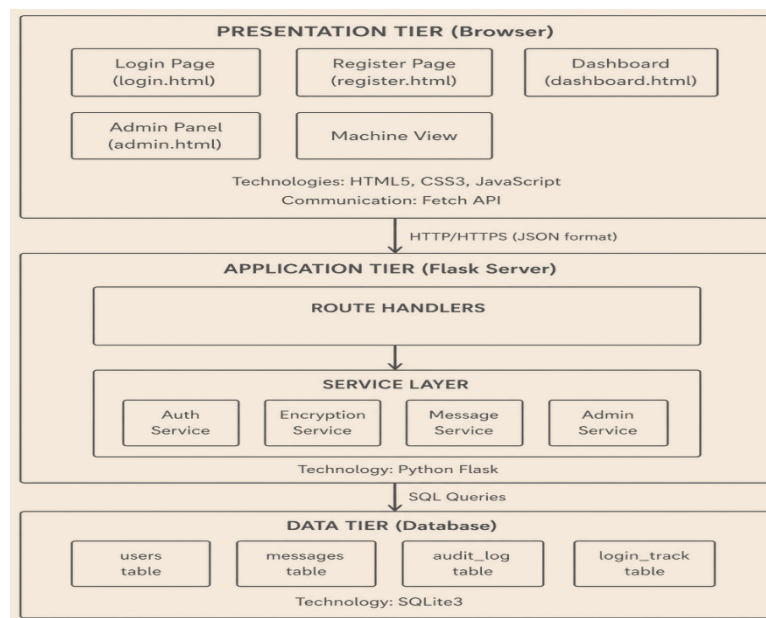
The system is a web-based secure messaging application designed for manufacturing environments where sensitive production data needs protection. When a sender types a message, the system encrypts it using the Fernet algorithm with a secret key before storing it in the database. When the receiver opens the message, the system decrypts it using the same key and displays the original message. Only the sender and receiver can read the message, ensuring end-to-end security. The system also supports role-based access with different permissions for Admin, Operator, QC Inspector, and Maintenance users.

2. System Architecture

2.1 Detailed Application Architecture

The system follows a 3-Tier Client-Server Architecture consisting of Presentation Tier (Frontend), Application Tier (Backend), and Data Tier (Database). This separation ensures modularity, security, and ease of maintenance.

Architecture Diagram:



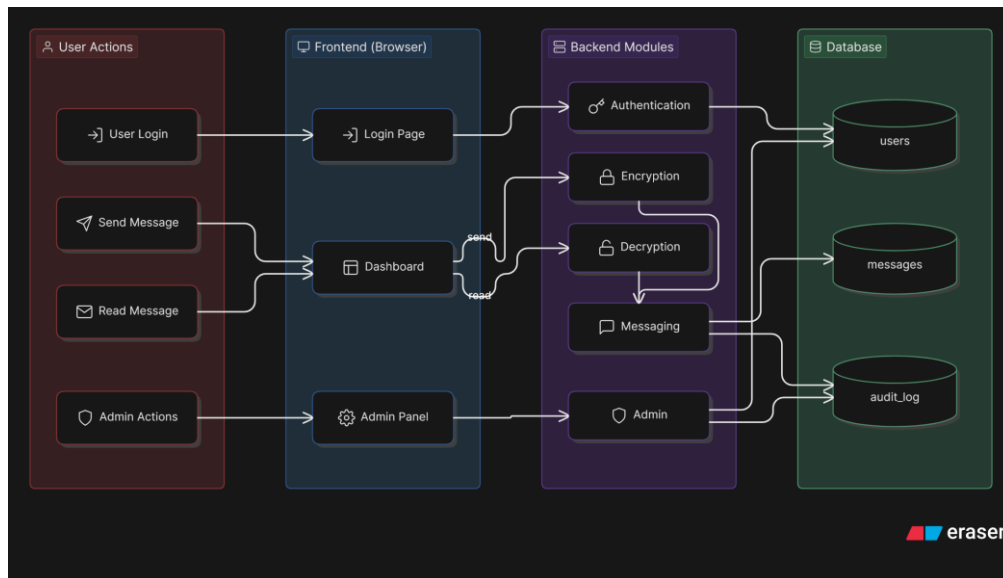
Tier Description Table:

Tier	Name	Technology	Responsibility
Tier 1	Presentation	HTML, CSS, JavaScript	User interface, form inputs, displaying results, sending API requests
Tier 2	Application	Python Flask	Business logic, encryption/decryption, request handling, session management
Tier 3	Data	SQLite3	Storing users, messages, audit logs, login tracking

2.2 Module Interaction Flow

This diagram shows how different modules within the system communicate with each other during various operations.

Module Interaction Diagram:



Interaction Flow Explanation Table:

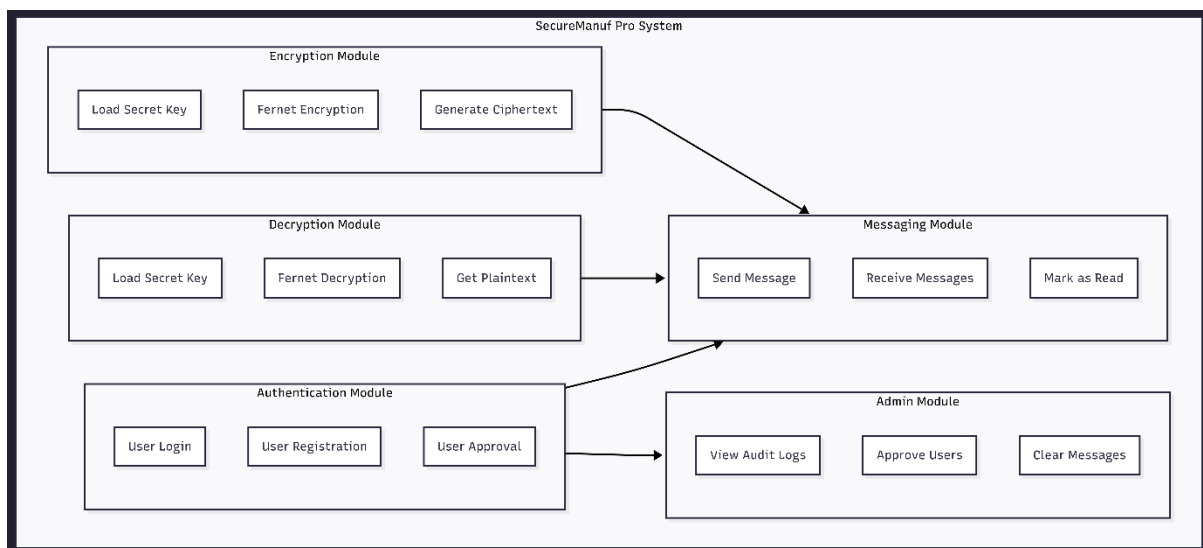
User Action	Frontend Page	Backend Module	Database Table	Description
Login	login.html	Authentication Module	users	Verifies credentials and creates session
Send Message	dashboard.html	Encryption + Messaging	messages, audit_log	Encrypts message and stores it
Read Message	dashboard.html	Decryption + Messaging	messages	Decrypts and displays message
Admin Approval	admin.html	Admin Module	users, audit_log	Approves or rejects pending users

3. Module Design

3.1 Module Overview

The system is divided into five core modules. Each module has a specific responsibility and works independently while communicating with other modules when needed. This modular design makes the system easy to develop, test, and maintain.

Module Overview Diagram:

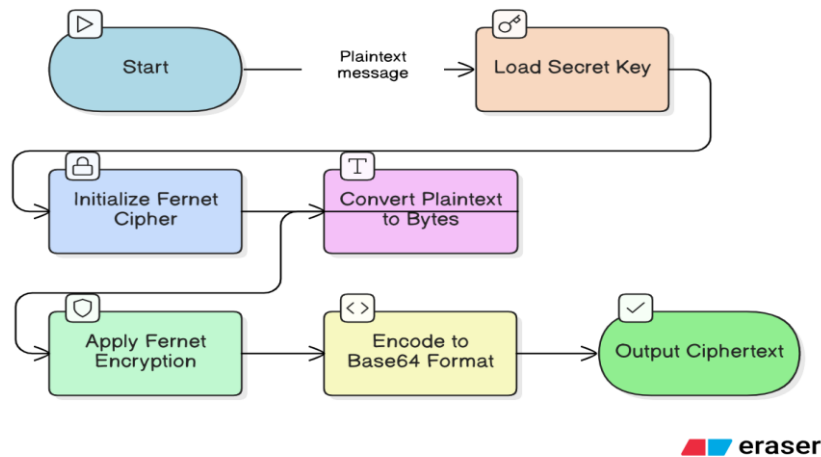


3.2 Authentication Module

This module manages user identity verification and access control. When a user logs in, the module checks the username against the database, verifies the password using hash comparison, confirms the account status is approved, and creates a session if all checks pass. For registration, the module validates that passwords match, ensures username uniqueness, hashes the password, and stores the user with pending status. For approval, the admin can change a user's status from pending to approved or rejected, and the action is logged in the audit trail.

3.3 Encryption Module

The Encryption Module is responsible for converting readable plaintext into unreadable ciphertext. This ensures that even if someone accesses the database, they cannot read the actual message content. The module uses the Fernet symmetric encryption algorithm which guarantees that only someone with the correct secret key can decrypt the message.

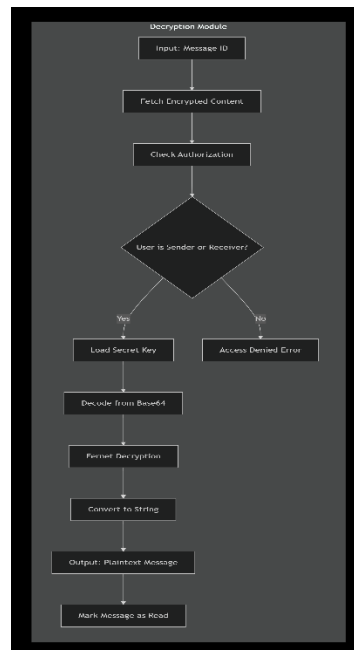


Process Steps	1. Load the secret key from key.key file 2. Initialize Fernet cipher with the loaded key 3. Convert the plaintext string to bytes 4. Encrypt the bytes using Fernet 5. Encode the encrypted bytes to Base64 format 6. Return the ciphertext as a string
---------------	--

3.4 Decryption Module

The Decryption Module is responsible for converting ciphertext back into original plaintext. This module includes an authorization check to ensure that only the sender or the intended receiver can decrypt and read the message.

Decryption Module Diagram:



Process Steps

1. Fetch the encrypted message from messages table by ID
2. Check if current user is sender or receiver
3. If not authorized, return access denied error
4. Load the secret key from key.key file
5. Decode the ciphertext from Base64 format
6. Decrypt the bytes using Fernet
7. Convert decrypted bytes back to string
8. Mark the message as read in database
9. Return the plaintext message

3.5 Messaging Module

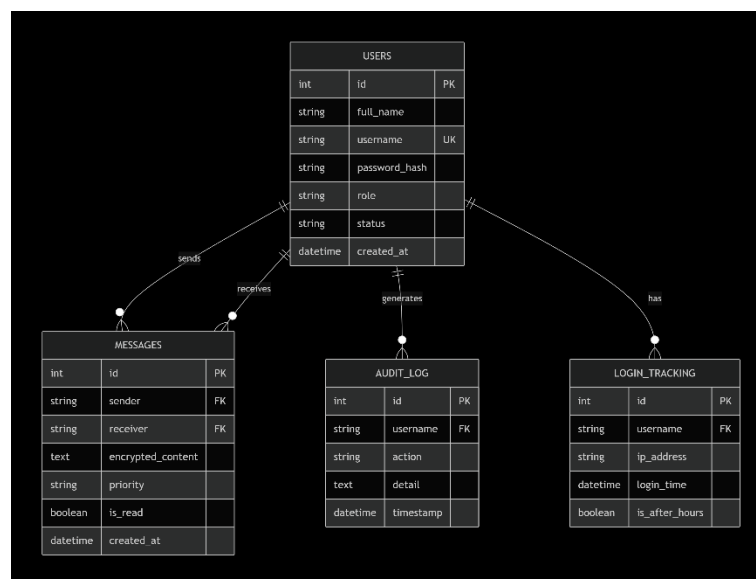
The Messaging Module handles all operations related to sending and receiving messages. It works closely with the Encryption and Decryption modules to ensure messages are stored securely and delivered only to authorized users

4. Database Design

4.1 Database Schema

The database consists of four tables that store all system data. The tables are designed to be simple yet complete, with proper relationships and constraints to maintain data integrity.

Entity Relationship (ER) Diagram:



4.2 Table Structures

Table 1: users

This table stores all user account information including credentials and role.

Column	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTO_INCREMENT	Unique identification number for each user
full_name	TEXT	NOT NULL	Complete name of the user

username	TEXT	NOT NULL, UNIQUE	Login username, must be unique across all users
password_hash	TEXT	NOT NULL	SHA-256 hashed password, never stored as plaintext
role	TEXT	NOT NULL	User role: admin, operator, qc, or maintenance
status	TEXT	DEFAULT 'pending'	Account status: pending, approved, or rejected
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Date and time when account was created

Table 2: messages

This table stores all encrypted messages sent between users.

Column	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTO_INCREMENT	Unique identification number for each message
sender	TEXT	NOT NULL	Username of the person who sent the message
receiver	TEXT	NOT NULL	Username of the person who should receive the message

encrypted_content	TEXT	NOT NULL	The message encrypted using Fernet, stored as Base64 text
priority	TEXT	DEFAULT 'normal'	Urgency level: critical, high, normal, or low
is_read	BOOLEAN	DEFAULT 0	0 means unread, 1 means read by receiver
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Date and time when message was sent

Table 3: audit_log

This table tracks all important actions performed by users for security and debugging.

Column	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTO_INCREMENT	Unique identification number for each log entry
username	TEXT	NOT NULL	Username of the user who performed the action
action	TEXT	NOT NULL	Type of action: LOGIN, LOGOUT, SEND, DECRYPT, APPROVE, etc.
detail	TEXT		Additional information about the action (optional)
timestamp	DATETIME	DEFAULT CURRENT_TIMESTAMP	Date and time when action occurred

4.3 Relationships

Definition:

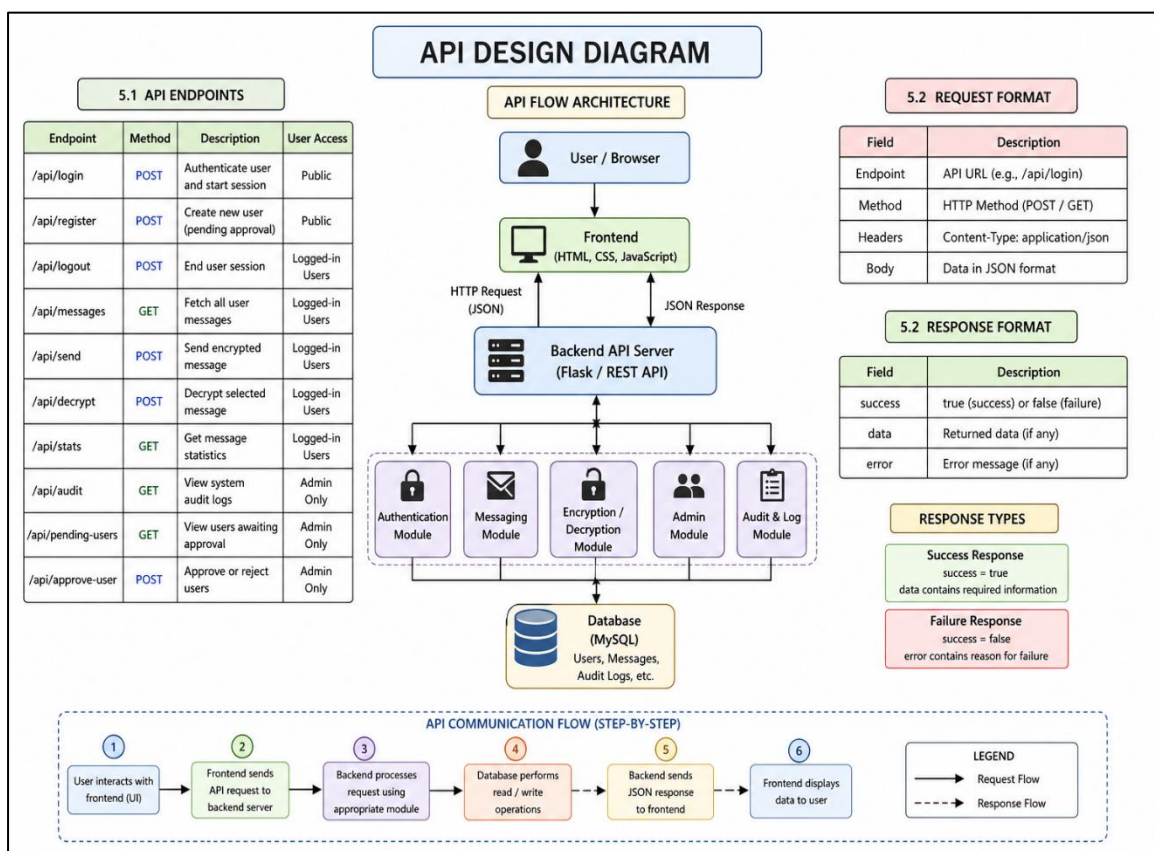
In a database system, relationships define how different tables are connected to each other. They help maintain data integrity and enable efficient data retrieval. Relationships are usually established using primary keys and foreign keys. Common types include one-to-one, one-to-many, and many-to-many relationships. In this system, relationships ensure proper linkage between users, messages, and tracking records while preserving important data for auditing and security purposes.

Relationship Table:

Relationship	Type	Description	Rules
users to messages (sender)	One-to-Many	One user can send many messages	When user is deleted, messages remain (for audit)
users to messages (receiver)	One-to-Many	One user can receive many messages	When user is deleted, messages remain (for audit)
users to audit_log	One-to-Many	One user can have many audit log entries	When user is deleted, logs remain (cannot be deleted)
users to login_tracking	One-to-Many	One user can have many login records	When user is deleted, login records remain

5. API Design

API (Application Programming Interface) is used for communication between frontend and backend using HTTP requests and JSON responses.



5.1 API Endpoints

Important API Endpoints Table

Endpoint	Method	Description	User Access
/api/login	POST	Authenticate user and start session	Public
/api/register	POST	Create new user (pending approval)	Public

/api/logout	POST	End user session	Logged-in Users
/api/messages	GET	Fetch all user messages	Logged-in Users
/api/send	POST	Send encrypted message	Logged-in Users
/api/decrypt	POST	Decrypt selected message	Logged-in Users
/api/stats	GET	Get message statistics	Logged-in Users
/api/audit	GET	View system audit logs	Admin Only
/api/pending-users	GET	View users awaiting approval	Admin Only
/api/approve-user	POST	Approve or reject users	Admin Only

Endpoint Classification

Category	APIs Included
Authentication	Login, Register, Logout
Messaging	Send, Messages, Decrypt
Dashboard	Stats
Admin Control	Audit, Pending Users, Approve User

5.2 Request and Response Format

Request Format Table

Field	Description	Example
Endpoint	API URL	/api/login
Method	HTTP method	POST / GET
Headers	Metadata (Content-Type)	application/json
Body	Data sent to server	Username, Password

Response Format Table

Field	Description	Example
success	Indicates operation result	true / false
data	Returned information	User details / messages
error	Error message (if failed)	Invalid credentials

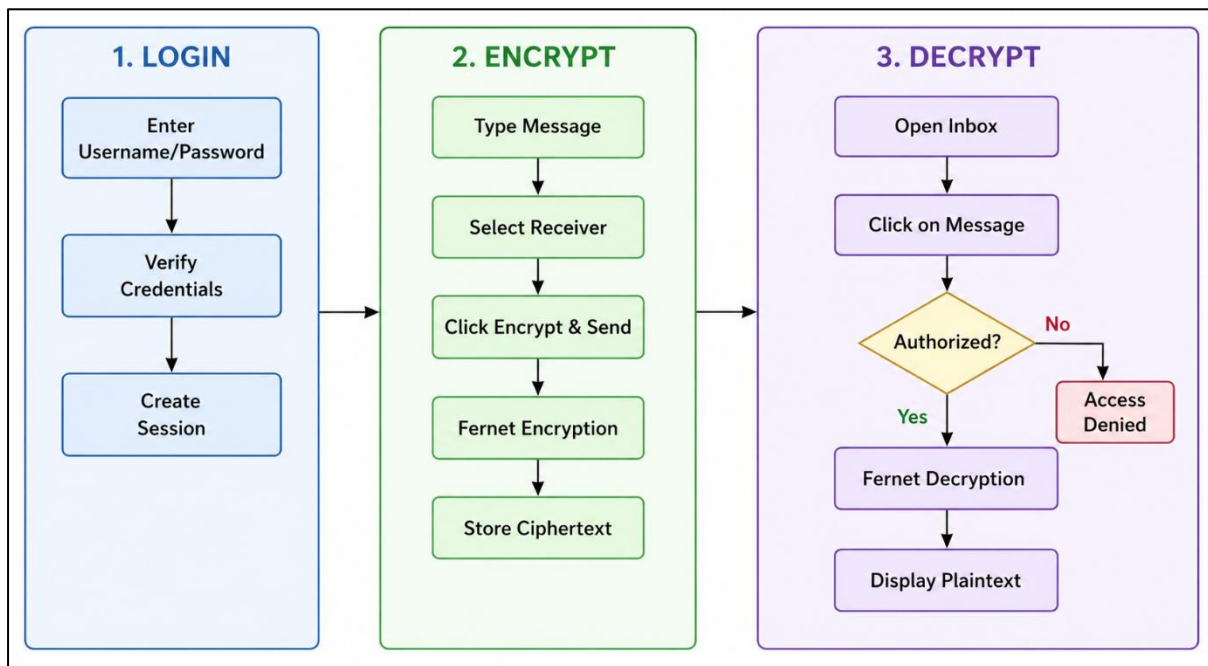
6. Process Flow

What is Process Flow?

Process flow is the sequence of steps that happen from the moment a user starts an action until the action is completed. It shows how data moves from one component to another and what happens at each stage. In this system, the main process flow covers three key actions: logging into the system, sending an encrypted message, and decrypting and reading that message.

6.1 System Workflow (Login → Encrypt → Decrypt)

Overall Flow Diagram:



1. Login Process

Login is the process of verifying a user's identity before allowing access to the system. The user enters their username and password. The system checks if the username exists in the database, then verifies if the password matches the stored hash. It also checks whether an admin has approved the account. If all checks pass, the system creates a session for the user and redirects them to the dashboard. If any check fails, an error message is shown. Every login attempt is recorded in the login tracking table for security.

2. Encrypt Message Process

Encrypt message is the process of converting a readable plaintext message into unreadable ciphertext before storing it. The user types a message, selects a receiver, and chooses a priority level. When they click send, the system loads the secret key, initializes the Fernet cipher, converts the plaintext to bytes, encrypts it, and encodes the result to Base64. Only this encrypted ciphertext is stored in the database. The original plaintext message is never saved anywhere, ensuring that even if someone accesses the database, they cannot read the actual message.

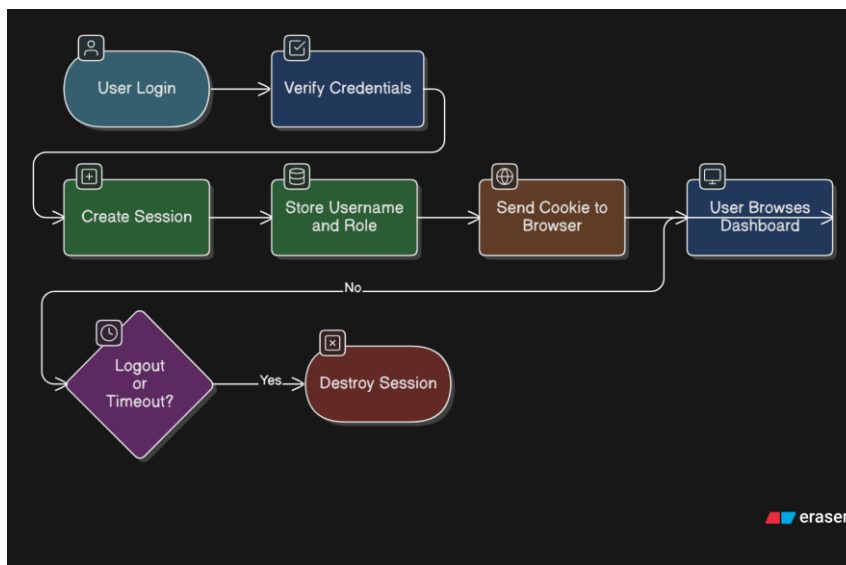
3. Decrypt Message Process

Decrypt message is the process of converting the stored ciphertext back into readable plaintext for authorized users. The receiver opens their inbox and clicks on a message. The system first checks if the user is authorized, meaning they must be either the sender or the intended receiver. If authorized, the system loads the same secret key, decodes the ciphertext from Base64, decrypts it using Fernet, and converts the bytes back to a string. The original message is then displayed on screen and automatically marked as read. If an unauthorized user tries to open the message, an "Access Denied" error is shown.

7. Session and Access Control

7.1 Session Handling

Session handling is the method used to keep a user logged in while using the application. In SecureManuf Pro, once a user logs in with valid and approved credentials, the server creates a session that stores details like username and role. A session cookie is sent to the browser, and it is used in every request so the user can access the dashboard, send messages, and view the inbox without logging in again. The session is stored on the server and automatically ends when the user logs out, closes the browser, or stays inactive for 30 minutes.

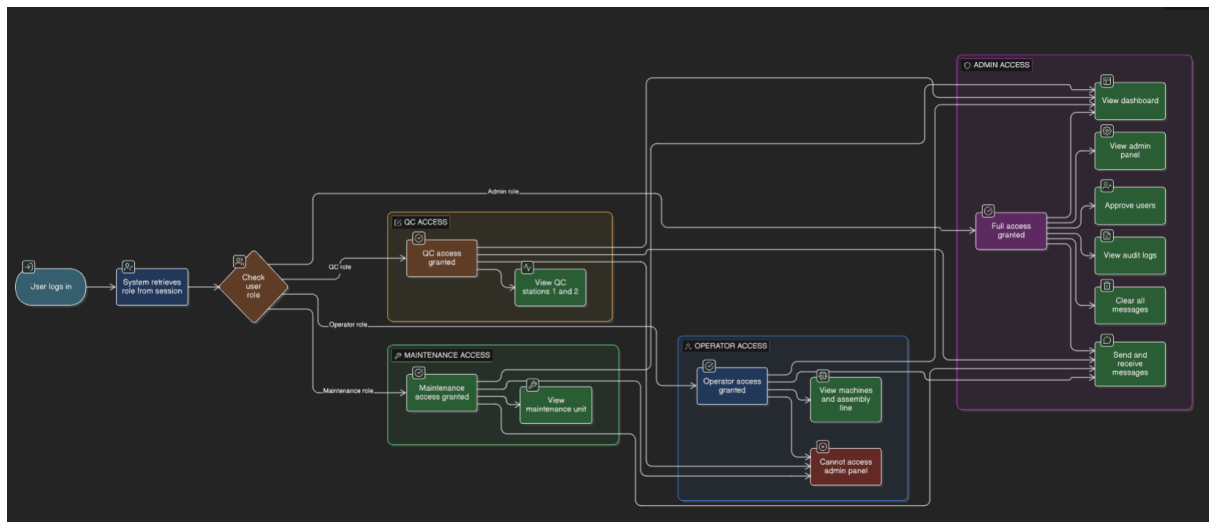


7.2 Role-Based Access Control

Definition:

Role-Based Access Control (RBAC) is a security approach where users are assigned specific roles, and each role has defined permissions. Instead of giving permissions directly to individual users, permissions are given to roles, and users inherit permissions from their assigned role. This makes it easy to manage access for many users. In this system, there are four roles: Admin, Operator, QC Inspector, and Maintenance. The Admin role has full access including approving users and viewing audit logs. Other roles have limited access based on

their job function. When a user tries to perform any action, the system checks their role from the session and allows or denies access accordingly



8. Error Handling and Validation

8.1 Error Handling

Definition: Error handling is the process of detecting when something goes wrong in the application and showing a helpful message to the user with the appropriate HTTP status code. In SecureManuf Pro, errors are displayed as toast notifications at the bottom right corner of the screen for 3 seconds.

Error Codes and Messages Table:

Error Code	HTTP Status	Message	When It Happens
AUTH-001	401 Unauthorized	Invalid username or password	User enters wrong login credentials
AUTH-002	403 Forbidden	Account pending approval	New user tries to login before admin approval
AUTH-003	401 Unauthorized	Session expired, please login again	User inactive for more than 30 minutes
AUTH-004	400 Bad Request	Username already exists	User tries to register with taken username
AUTH-005	400 Bad Request	Passwords do not match	Password and confirm password fields don't match
MSG-001	404 Not Found	Message not found	User tries to open a message that doesn't exist
MSG-002	403 Forbidden	Access denied	User tries to decrypt message not meant for them
MSG-003	400 Bad Request	Message cannot be empty	User tries to send a blank message
ADM-001	403 Forbidden	Admin access required	Non-admin user tries to access admin panel
SUCCESS	200 OK	Action completed successfully	Login, send, decrypt, or any action works fine

8.2 Validation Rules

Definition: Validation rules are checks performed on user input before processing it to ensure the data meets required standards.

Validation Rules Table:

Field	Rule	HTTP Status	Error Message
Username	Minimum 3 characters, letters and numbers only	400	Username must be at least 3 characters
Password	Minimum 4 characters	400	Password must be at least 4 characters
Full Name	Cannot be empty	400	Full name is required
Message	Cannot be empty, Maximum 5000 characters	400	Message cannot be empty
Confirm Password	Must exactly match Password field	400	Passwords do not match

9. References

<i>Reference</i>	<i>Source URL</i>	<i>Used For</i>
<i>Python Official Documentation</i>	https://docs.python.org/3/	Python programming language syntax and functions
<i>Flask Framework Documentation</i>	https://flask.palletsprojects.com/	Web framework for backend development
<i>Cryptography (Fernet) Documentation</i>	https://cryptography.io/en/latest/fernet/	Encryption and decryption implementation
<i>SQLite3 Documentation</i>	https://sqlite.org/docs.html	Database design and SQL queries
<i>HTML5 and CSS3 Documentation</i>	https://developer.mozilla.org/	Frontend user interface development
<i>JavaScript Documentation</i>	https://developer.mozilla.org/en-US/docs/Web/JavaScript	Client-side form validation and API calls
<i>Mermaid.js Documentation</i>	https://mermaid.js.org/	Creating flowcharts and diagrams
<i>OWASP Authentication Cheat Sheet</i>	https://cheatsheetseries.owasp.org/	Session management and security guidelines